Q.1] Define Generative AI and explain how it works with a suitable example.

Generative Artificial Intelligence (GenAI) is a type of AI that learns patterns from existing data and uses them to generate new content such as text, images, music, audio, and code.

Basic Idea

Existing Data → Learn Patterns → Generate New Content

Working of Generative AI

Step 1: Collect training data: Collect a large amount of text, images, audio, or other data.

Step 2: Train the model: A neural network learns patterns and relationships. Its weights are adjusted to reduce training error.

Step 3: Provide input: The user gives a prompt or other input, such as “Write a short story about a robot.”

Step 4: Generate content: The model uses the input and learned patterns to generate a response.

Step 5: Review the output: Check the result and change the prompt if needed.

Example

• Input: “Once upon a”

• A text-generating model predicts a likely next token, such as “time.”

• It repeatedly predicts additional tokens to complete the story.

A **token** is a small unit of text, such as a word or part of a word.

Final Flow

Training Data → Model Training → User Prompt → Generation → Review

Q.2] Explain the major families of Generative AI.

The major families of Generative AI differ in how they learn patterns and generate new content.

1) Variational Autoencoders - VAEs

• Contain an encoder and a decoder.

• The encoder represents input as a probability distribution in latent space.

• The decoder generates data from a sampled latent vector.

Example: Generating handwritten digit images.

2) Generative Adversarial Networks - GANs

• Contain a generator and a discriminator.

• The generator produces artificial samples.

• The discriminator distinguishes real samples from generated samples.

• Both improve through competition.

Example: Generating realistic face images.

3) Transformer-Based Generative Models / LLMs

• Use attention to capture relationships between tokens.

• Generative LLMs can produce text by predicting tokens sequentially.

• Used for text generation, summarization, translation, and coding.

4) Diffusion Models

• Noise is added to data during training.

• The model learns to reverse this process.

• Generation starts with noise and gradually creates a sample.

Example: Generating an image from a text description.

Family	Basic idea	Typical output
VAE	Encode → Sample → Decode	Images
GAN	Generator competes with discriminator	Images
Generative Transformer	Predict tokens using context	Text or code
Diffusion	Gradually remove noise	Images or audio

Q.3] Explain VAE, GAN, Transformer/LLM, and Diffusion models.

1) Variational Autoencoder - VAE

A **VAE** learns a probability distribution for latent representations of input data.

• **Encoder:** Produces mean and variance information.

• **Sampling:** Creates a latent vector.

• **Decoder:** Reconstructs or generates data.

Working: Encode the input → Obtain μ and σ → Sample z → Decode z → Calculate loss → Update weights.

$$z = \mu + \sigma \times \varepsilon, \quad \varepsilon \sim N(0, I)$$

Multiplication is element-wise. Training minimizes reconstruction loss and KL divergence.

Advantage: Structured latent space. **Limitation:** Basic image VAEs can produce blurry outputs.

2) Generative Adversarial Network - GAN

A **GAN** contains two neural networks that compete during training.

• **Generator:** Converts random noise into artificial data.

• **Discriminator:** Predicts whether data is real or generated.

Working: Generate fake samples → Compare real and fake samples → Train discriminator → Train generator → Repeat.

Advantage: Can generate sharp images. **Limitation:** Training can be unstable.

3) Transformer / Large Language Model - LLM

A **Transformer** uses attention to process relationships between tokens. An **LLM** is a large model trained on extensive language data. Many LLMs use Transformers.

Components: Token embeddings, positional information, attention layers, feed-forward layers, and output prediction layer.

Working: Tokenize prompt → Convert to vectors → Process context using attention → Predict next-token probabilities → Select token → Repeat.

Example: “The sun rises in the” → “east”.

Advantage: Performs many language tasks. **Limitation:** Can confidently generate incorrect information.

4) Diffusion Model

A **Diffusion Model** generates data by learning to reverse a gradual noising process.

• **Forward diffusion:** Gradually add noise to a real image.

• **Learn denoising:** Train a network to estimate noise or an equivalent denoising target.

• **Generation:** Start with random noise and repeatedly denoise it.

Advantage: Detailed, diverse images. **Limitation:** Repeated steps require computation.

Q.4] Explain the applications of Generative AI in art, music, and text generation.

1) Art and Image Generation

• Generates images from text descriptions.

• Creates illustrations, posters, and character designs.

• Edits selected parts of images.

• Helps artists explore different styles and ideas.

Example: “A mountain village covered in snow” → An image based on the description.

2) Music Generation

• Generates melodies and musical patterns.

• Creates background music.

• Produces accompaniments for existing melodies.

• Helps musicians explore variations in rhythm and style.

Example: “Create a calm piano melody” → A generated piano composition.

3) Text Generation

• Writes stories, emails, and articles.

• Summarizes long documents.

• Translates text.

• Generates chatbot responses.

• Helps create educational material.

Example: “Explain photosynthesis in simple words” → A simple explanation.

Advantages

• Saves time.

• Supports creativity.

• Produces multiple variations.

• Helps personalize content.

Q.5] Discuss the key challenges of Generative AI.

1) Data Quality

Models depend on training data. Incomplete or poor-quality data can produce poor outputs.

2) High Computational Cost

Large models require substantial memory and processing power. Training and running them can be expensive.

3) Incorrect Outputs / Hallucinations

Models may generate false facts or references. Fluent answers are not always correct.

4) Bias

Models can learn biases in training data and unfairly represent certain groups.

5) Output Control

Results may not completely follow a prompt. Consistency across long outputs can be difficult.

6) Evaluation

Creativity and usefulness are difficult to measure. One question can have many acceptable outputs.

7) Privacy and Security

Models may reveal memorized sensitive information. Malicious inputs can attempt to manipulate application behavior.

8) Model-Specific Training Problems

GANs may suffer from mode collapse. VAEs may suffer from posterior collapse. Some generation methods require many computational steps.

Example

A model generates a convincing answer containing an invented book reference. The answer looks correct but requires verification.

Q.6] Explain the ethical considerations of Generative AI.

Ethical considerations are principles used to ensure that Generative AI is developed and used responsibly.

1) Fairness

Avoid discriminatory outputs and evaluate performance across different groups.

2) Privacy and Consent

Protect personal and confidential information. Use data with appropriate permission.

3) Intellectual Property

Consider rights associated with training material and generated content. Avoid presenting someone else's work as original work.

4) Misinformation and Deepfakes

Generated text, images, and audio can mislead people. Disclose artificial content where necessary.

5) Transparency

Users should understand when they are interacting with AI and know its relevant limitations.

6) Accountability

People and organizations must take responsibility for AI use. Important outputs should receive suitable human review.

7) Impact on Work

Automation can change job roles. Training and reskilling can help people adapt.

8) Prevention of Harmful Use

Reduce misuse for fraud, impersonation, harassment, and other harmful activities.

Example

Generating a person's fake voice recording without permission can violate their privacy and mislead listeners.

Q.7] What are foundation models? Explain LLMs and compare foundation models with LLMs.

1) Foundation Model

A **Foundation Model** is trained on broad data and can be adapted to many different tasks.

• Learns reusable patterns and supports multiple downstream tasks.

• Can be adapted using prompting or fine-tuning.

• May process text, images, audio, or multiple data types.

Example: A broadly trained vision model adapted for several image recognition tasks.

2) Large Language Model - LLM

An **LLM** is a large neural network trained on extensive language data to process and generate language.

Applications: Question answering, text generation, summarization, translation, and code generation.

Text Input → Tokenization → Model Processing → Output Tokens → Response

Comparison

Basis	Foundation model	LLM
Meaning	Broadly trained model adaptable to many tasks	Large model focused on language
Scope	Broader category	Many LLMs belong to this category
Data	Text, images, audio, or combinations	Primarily language; some support other inputs
Tasks	Depends on model modality	Writing, summarization, translation, coding
Architecture	Can use different architectures	Commonly uses Transformers
Output	Text, images, audio, vectors, etc.	Primarily text or code
Adaptation	Prompting, fine-tuning, or other methods	Prompting, fine-tuning, or other methods

Key Relationship

Many LLMs are foundation models, but foundation models are not limited to language.

Q.8] Explain the development workflow for building a basic LLM application from the user's perspective.

An **LLM application** uses a language model to answer questions, generate text, summarize documents, or perform similar tasks.

Step 1: Identify the user's need: Decide what problem to solve, such as helping students understand notes.

Step 2: Define input and output: Input may be a question or document; output may be an explanation, summary, or answer.

Step 3: Select a suitable model: Consider quality, speed, cost, and privacy. Access it through an API or local deployment.

Step 4: Design the prompt: Specify the task, language, length, and format. Example: "Explain in five simple points with one example."

Step 5: Add relevant context: Provide needed information. Document applications can retrieve relevant passages using Retrieval-Augmented Generation (RAG).

Step 6: Build the interface: Provide a place to enter questions or upload documents and display answers.

Step 7: Connect and process: Send the prompt and context to the model, receive its response, and display it.

Step 8: Test the application: Check accuracy and usefulness. Test unclear questions, missing information, and inappropriate inputs.

Step 9: Deploy and improve: Make the application available, collect feedback, and improve prompts or retrieval.

User Flow

Open Application → Enter Question → Submit → Read Answer → Give Feedback

Example

A student uploads notes and asks, "Explain VAE." The application retrieves relevant content and generates a simple explanation.

Q.1] Explain the architecture and working of a Variational Autoencoder with a neat diagram.

A **Variational Autoencoder (VAE)** learns a probabilistic latent representation of data and uses it to reconstruct or generate samples.

Architecture

$$\text{Input } x \rightarrow \text{Encoder} \rightarrow \text{Mean } \mu \text{ and Variance } \sigma^2$$
$$\downarrow$$
$$\text{Sample } z = \mu + \sigma \times \varepsilon, \quad \varepsilon \sim N(0, I)$$
$$\downarrow$$
$$\text{Decoder} \rightarrow \text{Reconstructed Output } \hat{x}$$

1) Encoder

• Receives input data and extracts important features.

• Produces parameters of a latent probability distribution.

• Usually outputs mean μ and log variance $\log \sigma^2$.

2) Latent Space and Sampling

Latent space is a compact representation of data. A vector is sampled using:

$$z = \mu + \sigma \times \varepsilon, \quad \varepsilon \sim N(0, I)$$

Multiplication is element-wise.

3) Decoder

• Receives the latent vector and reconstructs the input.

• Can generate new samples from randomly sampled latent vectors.

Working

Step 1: Give input x to the encoder and obtain μ and variance information.

Step 2: Sample noise ε and calculate latent vector z.

Step 3: Pass z through the decoder to obtain reconstruction $\hat{x}$.

Step 4: Calculate reconstruction loss and KL divergence.

Step 5: Update encoder and decoder weights using backpropagation.

Step 6: Repeat over the training data.

Generating New Data

$$\text{Sample } z \sim N(0, I) \rightarrow \text{Trained Decoder} \rightarrow \text{New Sample}$$

Example: A VAE trained on handwritten digits can generate new digit-like images.

Q.2] Explain encoding and decoding in VAE. Describe the roles of μ, σ, ε, and z.

The handwritten symbols are slightly unclear; these notes use standard VAE notation.

1) Encoding

Encoding converts input data into the parameters of a latent probability distribution.

$$q_{\varphi}(z \mid x) = N(\mu, \text{diag}(\sigma^2))$$

- x is the input.
- The encoder learns mean and variance values.
- Each input has a distribution of possible latent representations.

2) Roles of the Symbols

Symbol	Meaning	Role
μ	Mean vector	Center of the latent distribution
σ	Standard deviation vector	Controls the spread
σ^2	Variance vector	Square of standard deviation
ε	Standard normal noise	Introduces randomness
z	Sampled latent vector	Input to the decoder

3) Reparameterization Trick

$$z = \mu + \sigma \times \varepsilon, \quad \varepsilon \sim N(0, I)$$

- Direct sampling makes ordinary backpropagation through sampling difficult.
- Reparameterization separates randomness into ε.
- Gradients can then flow through μ and σ.

If the Encoder Outputs Log Variance

$$v = \log \sigma^2 \quad \rightarrow \quad \sigma = \exp(v / 2)$$

$$z = \mu + \exp(v / 2) \times \varepsilon$$

4) Decoding

The decoder receives z and maps the compact representation back into data space.

$$\hat{x} = \text{Decoder}(z)$$

Example

If μ = 2, σ = 0.5, and ε = 1:

$$z = 2 + (0.5 \times 1) = 2.5$$

Input → Encode Distribution → Sample z → Decode → Reconstruction

Q.3] Explain the VAE loss function, reconstruction loss, KL divergence, and their significance.

$$L_{VAE} = L_{reconstruction} + L_{KL}$$

1) Reconstruction Loss

Measures how well the decoder reproduces the input by comparing x with its reconstruction $\hat{x}$. Smaller loss means a better reconstruction.

The exact form depends on the output distribution. One example is **Mean Squared Error (MSE)**:

$$L_{MSE} = (1 / n) \Sigma_i (x_i - \hat{x}_i)^2$$

Binary cross-entropy is another common choice for normalized digit images.

Significance

- Encourages preservation of useful input information.
- Helps the decoder produce outputs similar to the original data.

2) KL Divergence

Measures how different the encoder's latent distribution is from a chosen prior, usually N(0, I).

$$L_{KL} = D_{KL}(q_\varphi(z \mid x) \,||\, p(z))$$

For a diagonal Gaussian encoder and standard normal prior:

$$L_{KL} = \tfrac{1}{2} \Sigma_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$$

Significance

- Regularizes the latent representation.
- Encourages distributions to remain close to the prior.
- Helps the decoder generate useful samples from the prior.

3) Combined Effect and Balance

- Reconstruction loss → Preserve input details.
- KL divergence → Regularize the latent distribution.
- Too little KL influence can reduce the usefulness of prior sampling.
- Too much KL influence can cause poor reconstructions or posterior collapse.

$$\text{Weighted version: } L = L_{reconstruction} + \beta L_{KL}$$

For the basic VAE, $\beta = 1$.

Example

$$\text{Reconstruction loss} = 20, \text{ KL loss} = 3 \rightarrow \text{Total loss} = 23$$

Q.4] Explain VAE implementation for MNIST image generation using TensorFlow, Keras, and PyTorch.

MNIST contains handwritten digits from 0 to 9. Each grayscale image is 28 × 28 pixels (784 values). Keras can be used with TensorFlow; PyTorch provides an alternative implementation.

Step 1: Load and Preprocess

• Load images, convert to floating point, and divide pixel values by 255.

• Flatten to 784 values for a dense network. Labels are not needed for a basic unconditional VAE.

Step 2: Build the Encoder

Input: 784 → Dense: 256, ReLU → Two Branches: Mean and Log Variance

A latent size of 2 can be used for demonstration.

Step 3: Implement Sampling

$$\text{std} = \exp(0.5 \times \text{log_variance})$$
$$z = \text{mean} + \text{std} \times \text{standard_normal_noise}$$

Step 4: Build the Decoder

Latent z → Dense: 256, ReLU → Dense: 784, Sigmoid → Reshape: 28 × 28

Sigmoid produces pixel values between 0 and 1.

Step 5: Define Loss

Calculate reconstruction loss over each image and KL loss over latent dimensions. Average the total per-image loss over the batch.

$$L = L_{\text{reconstruction}} + L_{\text{KL}}$$

Step 6: Train

Take a batch → Encode → Sample z → Reconstruct → Calculate loss → Compute gradients → Update weights with Adam → Repeat for several epochs.

Task	TensorFlow with Keras	PyTorch
Dense layers	Dense	nn.Linear
Model structure	Keras model	nn.Module
Random noise	tf.random.normal	torch.randn_like
Gradients	tf.GradientTape	loss.backward()
Optimizer	Keras Adam	torch.optim.Adam

Step 7: Generate New Digits

Sample z from N(0, I) → Trained decoder → Reshape outputs to 28 × 28 → Display images.

Q.4 (continued)] Simplified VAE training pseudocode

This pseudocode describes the same training process in either framework; it is not a standalone executable program.

```
For each epoch:
For each batch:

mean, log_variance = encoder(batch)
std = exp(0.5 × log_variance)
epsilon = standard normal noise with the same shape as std
z = mean + std × epsilon
reconstructed = decoder(z)

reconstruction_loss = compare(batch, reconstructed)
kl_loss = 0.5 × sum over latent dimensions of
(mean² + exp(log_variance) - 1 - log_variance)

loss = batch_mean(reconstruction_loss + kl_loss)
Calculate gradients and update weights
```

TensorFlow / Keras Training

- Perform the forward pass and loss calculation inside tf.GradientTape.
- Compute gradients with respect to trainable encoder and decoder weights.
- Apply gradients using the optimizer.

PyTorch Training

- Clear previously stored gradients using optimizer.zero_grad().
- Perform the forward pass and compute loss.
- Call loss.backward() to calculate gradients.
- Call optimizer.step() to update weights.

Final Flow

MNIST → Normalize → Encoder → Sampling → Decoder → Loss → Training → New Digits

Q.5] Explain the major applications and challenges of VAE.

Applications

1) Image Generation: Generates images from sampled latent vectors, such as handwritten digits.

2) Data Augmentation: Produces synthetic training samples. Check their usefulness on the target task.

3) Anomaly Detection: Train mainly on normal data. Unusual samples may show high reconstruction error or unusual model scores, such as abnormal sensor patterns.

4) Representation Learning: Converts high-dimensional data into compact features for visualization and downstream tasks.

5) Image Denoising: Can be trained to reconstruct clean images from noisy inputs.

6) Missing Data Reconstruction: Can be adapted to estimate missing values or image regions.

7) Scientific Data Generation: Can generate candidate molecular or other structured representations. Candidates require further validation.

Challenges

1) Blurry Images: Basic VAEs may produce images with less sharp detail.

2) Posterior Collapse: The decoder may ignore z. The encoder distribution becomes close to the prior without retaining much useful input information.

3) Loss Balancing: Reconstruction quality and latent regularization compete.

4) Latent Dimension Selection: Too few dimensions lose information. Too many can make learning less effective or harder to interpret.

5) Distribution Assumptions: A simple Gaussian distribution may not capture complex patterns.

6) Evaluation: Low training loss does not guarantee realistic or useful samples.

Q.6] Numerical Problems - VAE

No specific numerical values were given in the question image. The following are worked practice examples.

1) Calculate the Latent Vector

Given: $\mu = [1, 2]$, $\sigma = [0.5, 0.2]$, $\varepsilon = [2, -1]$

$$z = \mu + \sigma \times \varepsilon$$
$$z_1 = 1 + (0.5 \times 2) = 2$$
$$z_2 = 2 + (0.2 \times -1) = 1.8$$
Answer: z = [2, 1.8]

2) Calculate Standard Deviation from Log Variance

Given: $\log \sigma^2 = \log 4$

$$\sigma = \exp(\tfrac{1}{2} \log \sigma^2) = \exp(\tfrac{1}{2} \log 4) = \sqrt{4}$$
Answer: σ = 2

3) Calculate Reconstruction Loss using MSE

Given: $x = [1, 0, 1]$, $\hat{x} = [0.8, 0.1, 0.6]$

$$L_{MSE} = (1 / n) \Sigma_i (x_i - \hat{x}_i)^2$$
$$= [(1 - 0.8)^2 + (0 - 0.1)^2 + (1 - 0.6)^2] / 3$$
$$= (0.04 + 0.01 + 0.16) / 3$$
Answer: L_{MSE} = 0.07

4) Calculate KL Divergence

Given: $\mu = 1$ and $\sigma^2 = 1$ for a one-dimensional latent distribution.

$$L_{KL} = \tfrac{1}{2} (\mu^2 + \sigma^2 - 1 - \log \sigma^2)$$
$$= \tfrac{1}{2} (1 + 1 - 1 - 0)$$
Answer: L_{KL} = 0.5

Here, log means natural logarithm.

5) Calculate Total VAE Loss

Given: reconstruction loss = 12 and KL loss = 2.

Standard VAE: $L = 12 + 2 =$ **14**

If $\beta = 0.5$: $L = 12 + (0.5 \times 2) =$ **13**

Important: Check whether the question requires a sum or mean loss, and whether it gives variance, standard deviation, or log variance.

Q.1] Explain the architecture and working of a GAN with a neat diagram.

A **Generative Adversarial Network (GAN)** contains a generator and a discriminator trained through competition.

Architecture

Random Noise z → Generator G → Fake Samples
↓
Real Dataset → Discriminator D → Real/Fake Score

The discriminator receives both real and fake samples. Its feedback helps train the generator.

1) Generator

Takes random noise and produces artificial samples. Learns to make the discriminator classify generated samples as real. **$\hat{x} = G(z)$**

2) Discriminator

Receives real and generated samples. For a standard sigmoid discriminator, **D(x) is between 0 and 1**: near 1 means predicted real; near 0 means predicted fake.

Working

Step 1: Initialize: Initialize generator and discriminator weights.

Step 2: Generate fake data: Sample noise z and compute G(z).

Step 3: Train discriminator: Use real samples with label 1 and generated samples with label 0. Update discriminator weights.

Step 4: Train generator: Pass generated samples through the discriminator. Encourage high “real” scores. Update generator weights while keeping discriminator weights fixed.

Step 5: Repeat: Alternate the two updates until a suitable stopping point.

Loss Function

The original GAN objective is:

$$\min_G \max_D V(D, G) = E_{x \sim p_data}[\log D(x)] + E_{z \sim p(z)}[\log(1 - D(G(z)))]$$

Discriminator: Improve real/fake identification. **Generator:** Make generated samples appear real.

A common practical generator loss is:

$$L_G = -E_z[\log D(G(z))]$$

Example: Generate artificial handwritten digits and improve them using discriminator feedback.

Advantages: Sharp images and fast generation after training.

Limitations: Unstable training; mode collapse (limited sample variety); careful network balancing required.

Q.2] Compare VAE and GAN.

Basis	VAE	GAN
Full form	Variational Autoencoder	Generative Adversarial Network
Architecture	Encoder and decoder	Generator and discriminator
Main idea	Learn a probabilistic latent representation and reconstruct data	Generate samples through competition
Training process	Jointly optimize reconstruction and regularization	Alternate generator and discriminator updates
Loss function	Reconstruction loss + KL divergence	Adversarial losses
Image quality	Basic VAEs often produce smoother or blurrier images	Often produces sharper images
Latent space	Encoder distributions are regularized toward a prior	Generator maps a sampled prior to data
Input encoding	Has a built-in encoder	Basic GAN has no built-in encoder
Reconstruction	Directly trained to reconstruct inputs	Basic GAN is not trained for input reconstruction
Training stability	Usually more stable	Can be unstable
Diversity	Often provides useful coverage; depends on training	Can lose diversity through mode collapse
Advantages	Reconstruction, representation learning, structured latent space	Sharp samples and fast generation
Limitations	Blurry outputs, posterior collapse, loss balancing	Mode collapse, unstable training, difficult evaluation
Applications	Reconstruction, anomaly detection, representation learning	Image synthesis and image translation

Easy to Remember

VAE: Encode → Sample → Decode
GAN: Generate → Discriminate → Improve